

Advance Functions

1. Lambda Function:

Definition: A lambda function is an anonymous function in Python, defined using the `lambda` keyword. It is often used for short, one-time operations.

```
lambda arguments: expression
```

```
add = lambda x, y: x + y
result = add(3, 5)
print(result) # Output: 8
```

Filter Function:

Definition: the `filter` function is an advanced function that is used to filter elements from an iterable (e.g., a list) based on a specified function for which the provided function returns `True`

Syntax:
`filter(function, iterable)`

```
# Define a function to check if a number is even
def is_even(x):
    return x % 2 == 0

# List of numbers
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# Use the filter function to get even numbers
even_numbers = filter(is_even, numbers)

# Convert the result to a list and print it
print(list(even_numbers))
```

map Function

The `map` function takes a function and one or more iterables as input, applies the function to each corresponding element in the iterables, and returns an iterator of the results.

Syntax:

```
map(function, iterable, ...)
```

```
# Define a function that squares a number
def square(x):
    return x ** 2

# List of numbers
numbers = [1, 2, 3, 4, 5]

# Use map to square each number in the list
squared_numbers = map(square, numbers)

# Convert the result to a list and print it
print(list(squared_numbers))
```

Use cases:

- Transforming data: Apply a function to each element of a list to transform it into another form.
- Performing element-wise operations on multiple iterables simultaneously.
- Generating new sequences based on existing data.

reduce Function

The `reduce` function is a built-in function in Python's `functools` module. It is used for performing a rolling computation on sequential pairs of elements in an iterable, reducing the iterable to a single accumulated result. The `reduce` function takes two arguments: the function to perform the computation and the iterable to be reduced.

Syntax:

```
from functools import reduce
reduce(function, iterable[, initializer])#initializer--option
```

```

from functools import reduce

# Define a function to add two numbers
def add(x, y):
    return x + y

# List of numbers
numbers = [1, 2, 3, 4, 5]

# Use reduce to calculate the sum of the numbers
sum_of_numbers = reduce(add, numbers)
print("Sum of numbers:", sum_of_numbers) # Output: 15 (1 + 2 + 3 + 4 + 5)

# from functools import reduce
# # reduce(function, sequence)
# d=reduce(lambda a,b: a+b, [23,21,45,98])
# print(d)

```

Generator function

Generator-Function : A generator-function is defined like a normal function, but whenever it needs to generate a value, it does with the yield keyword rather than return. If the body of a def contains yield, the function automatically becomes a generator function.

```

# A Python program to demonstrate use of
# generator object with next()

# A generator function
def simpleGeneratorFun():
    yield 1 # pause or hold
    yield 2 # pause or hold
    yield 3 # pause or hold

# x is a generator object

```

```
x = simpleGeneratorFun()

# #Iterating over the generator object using next
print(x.__next__()) # In Python 3, __next__()

# print(x.__next__())
# print(x.__next__())
```

Difference b/w return and yield

The yield statement is responsible for controlling the flow of the generator function.

It pauses the function execution by saving all states and yielded to the caller. Later it resumes execution when a successive function is called. We can use the multiple yield statement in the generator function.

The return statement returns a value and terminates the whole function and only one return statement can be used in the function.